When are Custom Drivers Needed for vAttention?

Team 3
Joshua Frank, Londy Jean, Beethoven Marhone, Kyle Merritt

Abstract—In the context of large language model inference, the KV cache stores the key and value activations needed to reuse prior attention state during autoregressive decoding, so reducing its memory footprint directly improves throughput, batch size, and the maximum practical context length. vAttention was proposed as an alternative to vLLM's PagedAttention to preserve contiguous virtual memory for the KV cache while relying on GPU demand paging for on-demand physical allocation. This let the system to reuse unmodified high-performance kernels, including FlashAttention, instead of requiring paging-aware attention kernels and explicit block-table management in the serving stack. The vAttention paper reported up to 1.97× faster token generation than vLLM and up to 3.92× and 1.45× faster prompt processing than the PagedAttention variants of FlashAttention and FlashInfer. However, under the stock NVIDIA driver path, this design is constrained by a 2 MB page granularity, which can still create non-trivial tail waste. The vAttention project mitigated this waste by implementing a custom open source NVIDIA driver. This project asks whether the custom vAttention driver path is still necessary once context lengths become large. We characterize tail-waste fragmentation under a fixed 2 MB page size for three attention families: multi-head attention (MHA), grouped-query attention (GQA), and multi-head latent attention (MLA). Our evaluation covers Qwen-14B (MHA), Mistral-Nemo-12B (GQA), DeepSeek-V2-Lite (MLA), and a novel synthetic Mistral MLA implementation that we added to the vAttention codebase to enable a controlled GQA versus MLA comparison on the same backbone. The answer to the primary question is conditional. Some models can reach context ranges where 2 MB tail waste becomes insignificant, but architectural improvements, like GQA and MLA, also change the fragmentation curve and can push that crossover point far to the right. This ambiguity motivates our second contribution: an analytical model that predicts tail-waste fragmentation from model parameters and context length. The measured runs and side-by-side analytical comparisons show that these predictions closely match observed sawtooth behavior, giving system implementors a practical way to decide whether deploying the custom vAttention driver is worthwhile for their target context range and model architecture.

I. Introduction and Background

Large language model serving systems increasingly rely on paged KV-cache management to scale long-context inference. Paged designs reduce copying and simplify memory reuse, but they introduce tail waste: the final mapped page of a request is rarely perfectly full. vAttention addresses this problem with a custom driver path that can reduce the page size from 2 MB to 64 KB. The practical deployment question is whether that additional systems complexity is still justified as context windows get longer.

Our primary research question is:

> Do we still need the vAttention custom driver that reduces page size from 2 MB to 64 KB, even at long context lengths?

We began from the intuition in our proposal that 2 MB pages might be "good enough" for sufficiently long contexts, especially for dense architectures such as MHA. That intuition was only partly correct. Once we compared multiple attention architectures, the answer became more nuanced. Efficient attention layouts reduce bytes per token, therefore under a fixed 2 MB page size the number of tokens that fit into each page increases. As a result, the tail-waste percentage decreases more slowly as context grows, and the context length required to push fragmentation below a threshold such as 5% can become much larger than we originally expected.

This led to our second research question:

> How can a user know in advance when 2 MB pages will cause significant tail waste for a given model and target context range?

The contribution of this project is therefore twofold. First, we built an experimental pipeline that characterizes tail waste across architectures. Second, we derived and validated analytical formulas that predict the fragmentation curve from model parameters and context length. A further novel systems contribution is that we added an MLA implementation path to the vAttention codebase used in this project, including a synthetic Mistral MLA path. That extension made the controlled GQA versus MLA comparison possible.

II. Design and Implementation

Our implementation extends the vattention repository in three main directions.

A. Measurement pipeline

We added an end-to-end experimental pipeline that starts a model server, sends one request at a time with exact prompt token counts, records per-request metrics, and generates plots from the resulting CSVs. The pipeline writes host-visible outputs under server-output/ and derived figures under server_plots/. The resulting measurements are based on a single active request with max_tokens=1, which isolates allocator tail waste from unrelated batch interactions.

B. MLA support and synthetic Mistral MLA

The repository state we started from only supported dense attention layouts such as MHA and GQA. During

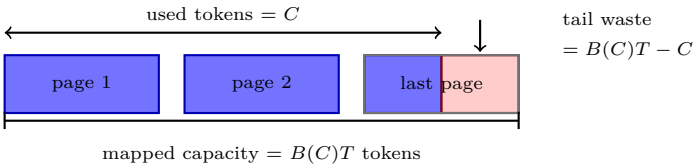


Fig. 1: Tail waste under paged KV allocation. A request with context length C maps $B(C) = \lceil C/T \rceil$ pages, each holding T tokens, so the unused capacity in the final mapped page is $B(C)T - C$.

this project we added MLA support to the vAttention codebase from scratch, including the cache-layout abstractions, runtime integration, model scaffolding, and DeepSeek-V2-Lite serving path needed to exercise a real MLA model. We then extended that work with MistralMLAForCausalLM and the supporting loader and configuration logic necessary to run a synthetic MLA version of Mistral-Nemo-12B. That second step enabled a controlled backbone-matched GQA versus MLA comparison, while the broader MLA implementation itself is one of the project's primary novel systems contributions.

C. Analytical tail-waste model

The important distinction from our original proposal is that the runtime metric is a tail-slack percentage in mapped token capacity, not a fixed-byte-waste ratio across the whole cache. Let C be context length in tokens, let T be the allocator's number of cached tokens per 2 MB page, and let $B(C) = \lceil C/T \rceil$ be the number of blocks (pages) allocated for that request. Figure 1 shows the resulting mapped capacity, used capacity, and tail slack. The exact fragmentation law used by the runtime is

$$F_{\text{exact}}(C) = 100 \cdot \frac{\lceil C/T \rceil T - C}{\lceil C/T \rceil T}. \tag{1}$$

This produces the empirical sawtooth shape directly. For a smooth upper envelope, we use

$$F_{\text{worst}}(C) \approx 100 \cdot \frac{T}{C + T}. \tag{2}$$

We also retain the average approximation from the original intuition that the final page is half full on average:

$$F_{\text{avg}}(C) \approx 100 \cdot \frac{T/2}{C + T/2}. \tag{3}$$

In the paper figures, we use F_{worst} as the main overlay because it cleanly predicts the vertical scale of the measured peaks, while F_{exact} is used for side-by-side validation.

The architecture-dependent quantity is T:

$$T = \left\lfloor \frac{P}{S_{\text{page-token}}} \right\rfloor, \tag{4}$$

where $P = 2{,}097{,}152$ bytes is the physical page size and $S_{\text{page-token}}$ is the per-token page-buffer footprint. The floor indicates that a 2 MB page does not always divide evenly into an integer number of token slots, so there can be a small fixed packing loss per mapped page. We note that effect but do not model it separately in the fragmentation curves, because the runtime quantity studied in this project and tied to our research question is tail slack in the final mapped page as context grows. The terms in $S_{\text{page-token}}$ are not fitted constants. They come directly from the trained model configuration and therefore from the model architecture itself. In other words, our analytical predictor uses architectural parameters already exposed by the model, rather than reverse-engineering them from the measurements.

For dense KV layouts (MHA/GQA/MQA),

$$T_{\text{dense}} = \left\lfloor \frac{P}{H_{\text{kv,loc}} D b} \right\rfloor, \tag{5}$$

where $H_{\text{kv,loc}}$ is the local KV-head count on one tensor-parallel worker, D is the per-head hidden dimension, and b is the number of bytes per stored element, e.g., $b = 2$ for FP16 or BF16 KV storage. In MHA, $H_{\text{kv,loc}}$ equals the local attention-head count because every head has its own key and value vectors. In GQA, multiple query heads share the same KV heads, so $H_{\text{kv,loc}}$ is smaller, which changes the number of tokens that fit in a page. For MLA,

$$T_{\text{MLA}} = \left\lfloor \frac{P}{(r_{\text{kv}} + d_{\text{rope}}) b} \right\rfloor. \tag{6}$$

Here r_{kv} is the KV latent rank and d_{rope} is the dimensionality of the RoPE-encoded key component retained by the MLA cache. These are also architectural parameters fixed by the trained model design and reported in the model configuration. The formulas explain why our original MHA-only proposal was incomplete: the percentage fragmentation curve is controlled by tokens per page, not by total cache bytes per token alone. GQA lowers the local KV-head count and therefore increases T relative to MHA, while MLA replaces dense K/V storage with a latent KV rank and RoPE slice, making T depend on $(r_{\text{kv}} + d_{\text{rope}})$.

We define 5% wasted memory as insignificant waste. This threshold is intentionally arbitrary and workload dependent, but it provides a concrete decision boundary. Under the worst-case envelope, the threshold occurs when

$$F_{\text{worst}}(C) \leq 5 \iff C \geq 19T. \tag{7}$$

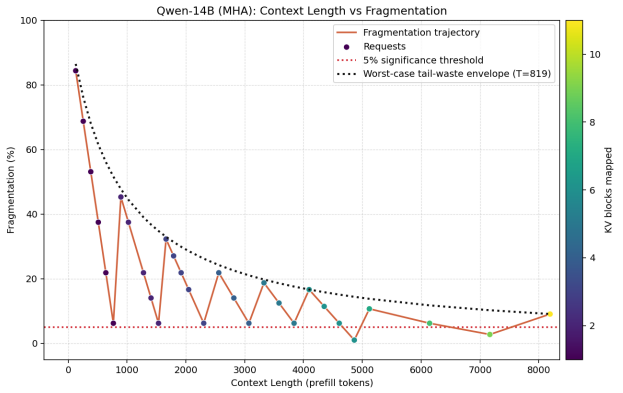
III. Evaluation and Results

We measured three architectural families under a fixed 2 MB page size:

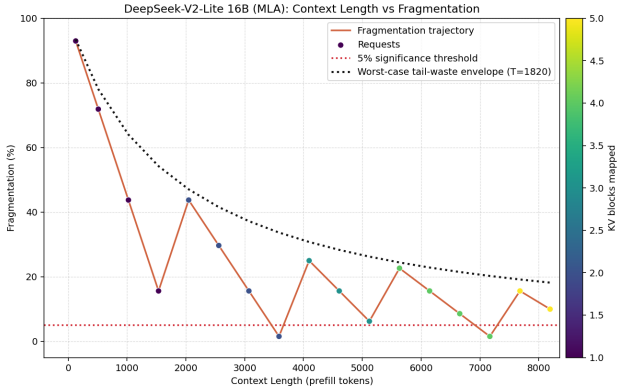
- Qwen-14B as MHA,
- Mistral-Nemo-12B as GQA,
- DeepSeek-V2-Lite as real MLA,
- synthetic Mistral-Nemo-12B MLA for a backbone-matched GQA versus MLA comparison.

The measured tokens per page were:

- Qwen-14B (MHA): $T = 819$
- Mistral-Nemo-12B (GQA): $T = 4096$
- DeepSeek-V2-Lite (MLA): $T = 1820$



(a) Qwen-14B (MHA).



(b) DeepSeek-V2-Lite (MLA).

Fig. 2: Expected MHA versus MLA contrast in tail-waste fragmentation. DeepSeek-V2-Lite uses a more memory-efficient MLA cache layout than Qwen-14B, but because that efficiency increases tokens per 2 MB page, its fragmentation envelope falls more slowly with context length. The result is the expected tradeoff: lower bytes per token, but a farther-right context threshold for making 2 MB pages insignificant.

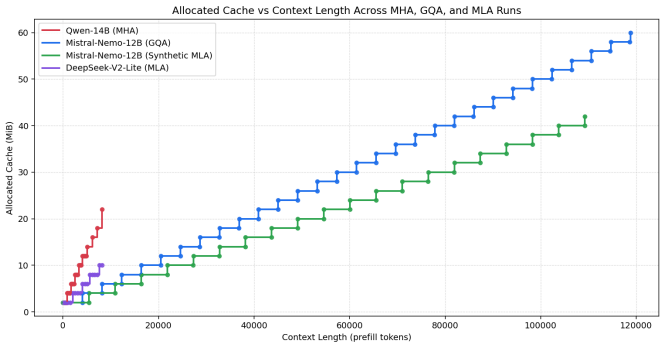


Fig. 3: Allocated cache versus context length across the four measured runs. MHA and DeepSeek grow faster in allocated cache than the two Mistral variants, while the synthetic MLA conversion yields the smallest long-context growth among the Mistral pair.

- Mistral-Nemo-12B synthetic MLA: $T = 5461$

Our first cross-architecture comparison behaved as expected. Figure 2 shows that the real MLA model, DeepSeek-V2-Lite, is more memory efficient than the dense MHA baseline, Qwen-14B, yet its larger tokens-per-page value pushes the tail-waste curve to decay more slowly. In other words, MLA can reduce allocated cache growth while simultaneously requiring a longer context to cross the 5% fragmentation threshold. This is exactly the tradeoff predicted by the analytical model.

The next comparison was less intuitive because MLA is generally intended to reduce KV-cache memory relative to denser layouts such as GQA, so one might expect DeepSeek-V2-Lite to outperform Mistral-Nemo-12B in allocated cache growth. Figure 3, however, shows the opposite trend over our measured range: the two Mistral runs were more favorable than both Qwen-14B and DeepSeek-V2-Lite. This result cautioned us against treating the DeepSeek-versus-Mistral observation as a pure MLA-versus-GQA experiment, because the backbones differ in many other architectural respects.

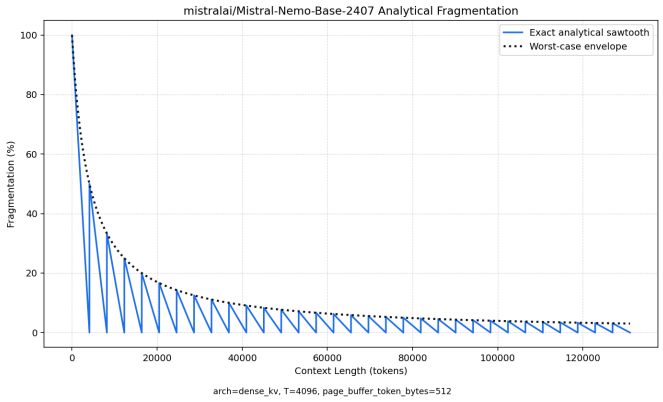
The synthetic Mistral MLA path resolved the ambiguity from the DeepSeek-versus-Mistral comparison. Figure 5 provides the apples-to-apples experiment we actually wanted: GQA and MLA on the same backbone. In that controlled setting, MLA again reduces allocated bytes while delaying the point at which 2 MB tail waste becomes negligible. This is the clearest evidence that the relevant fragmentation behavior is governed by tokens per page rather than by the MLA label.

Figure 4 addresses the second research question from the analytical side: using only model geometry, it predicts the full sawtooth structure for both Mistral configurations and makes clear why MLA shifts the curve farther right. Figure 5 then shows that the measured runs follow the same story in practice. Quantitatively, the exact analytical prediction error was effectively zero for the short Qwen and DeepSeek runs and remained small for the long Mistral runs: the mean absolute error was approximately 0.06 percentage points for Mistral GQA and 0.12 percentage points for synthetic Mistral MLA. That is sufficiently accurate for deployment-oriented decision support.

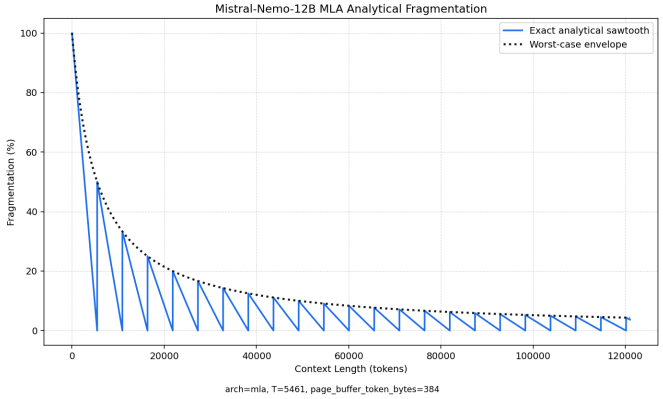
Using the worst-case threshold $C_{5\%} = 19T$, we obtain the following breakpoints:

Model	T	$C_{5\%}$ (tokens)
Qwen-14B (MHA)	819	15,561
DeepSeek-V2-Lite (MLA)	1820	34,580
Mistral-Nemo-12B (GQA)	4096	77,824
Mistral synthetic MLA	5461	103,759

These thresholds explain the paper's central conclusion. If a provider offers a very long maximum context, then even 2 MB pages may eventually become acceptable. For example, DeepSeek-V2-Lite advertises a much larger model context than the 32 k cap used in our measured



(a) Exact analytical sawtooth for Mistral-Nemo-12B (GQA).

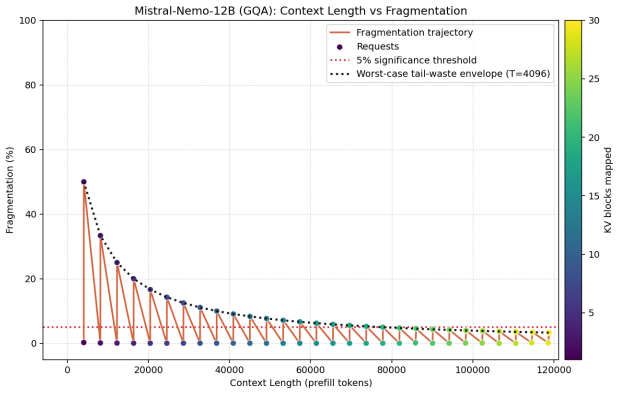


(b) Exact analytical sawtooth for synthetic Mistral-Nemo-12B MLA.

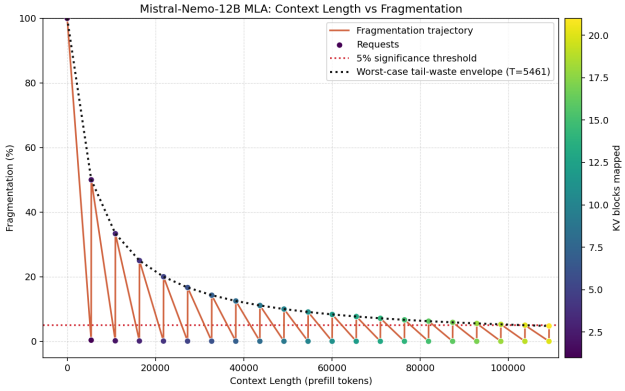
Fig. 4: Analytical fragmentation predictions for the backbone-matched Mistral runs. The model predicts the full sawtooth structure from architecture and context length alone, and it also makes the GQA-versus-MLA shift in tokens per page visually explicit before looking back at the measured runs.

server run, and its analytical threshold occurs around 34.6 k tokens. In that regime, 2 MB tail waste can plausibly become insignificant. But the controlled Mistral experiment shows the opposite pressure just as clearly: as attention becomes more memory efficient, each 2 MB page holds more tokens, and the 5% threshold shifts farther right. The synthetic Mistral MLA run makes this especially clear. Although MLA reduces total memory footprint, it pushes the worst-case insignificance threshold to roughly 103.8 k tokens, very close to the end of the long-context range we were able to reach experimentally.

Therefore the answer to the primary research question is it depends. A user can sometimes get away with the stock 2 MB granularity, but the decision depends on architecture and target context range. The best-case scenario for memory efficiency is still the combination of a small page size and an efficient attention layout. Efficient attention does not remove the value of smaller pages; in some cases it increases that value by delaying the point at which tail



(a) Mistral-Nemo-12B (GQA).



(b) Synthetic Mistral-Nemo-12B MLA.

Fig. 5: Backbone-matched GQA versus MLA comparison on Mistral-Nemo-12B. Once the backbone is held fixed, the expected tradeoff reappears clearly: the synthetic MLA variant uses less cache overall, but its larger tokens-per-page value shifts the fragmentation curve farther right than the GQA baseline.

waste becomes negligible.

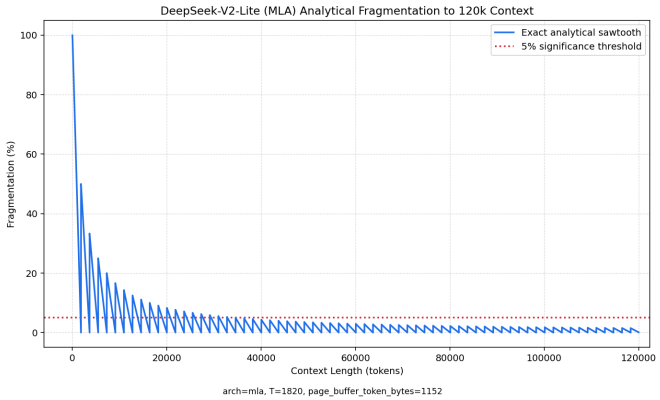


Fig. 6: Analytical DeepSeek-V2-Lite fragmentation extended to 120 k tokens. Because the validated model gives $T = 1820$ and $C_{5\%} = 19T = 34{,}580$, the predicted curve falls below the 5% insignificance threshold shortly after 40 k tokens, supporting the conclusion that sufficiently long contexts can make 2 MB tail waste effectively negligible for this model family.

IV. Conclusion

This project set out to determine whether the vAttention custom small-page driver is still necessary at long context lengths. Our answer is conditional rather than absolute. For some models, such as DeepSeek-V2-Lite, our validated analytical model shows that sufficiently long contexts can make 2 MB tail waste effectively insignificant. For others, especially when memory-efficient attention increases the number of tokens packed into each page, significant waste can persist deep into the usable context range. The controlled Mistral GQA versus synthetic Mistral MLA comparison demonstrates this complication clearly.

The more useful outcome of the project is not a one-size-fits-all rule, but a predictive method. We showed that tail waste can be predicted from model parameters and context length with high accuracy. That lets users estimate average or worst-case fragmentation over their intended operating range and decide whether the operational overhead of installing custom open-source NVIDIA drivers is justified for their workload.

In addition to answering the research questions, we contributed new implementation capability to the repository by adding MLA support and a synthetic Mistral MLA path, which enabled a novel experimental comparison that was not previously available in this codebase.

Appendix

Grader Verification Guide. After finishing the report, graders can use the repository's verification guide to inspect the implementation additions and run the recommended smoke checks without needing to reproduce the full GPU-dependent pipeline immediately. The public project repository is https://github.com/Anodyine/vattention, and the verification guide is located in that repository at docs/grader-verification-guide.md.

The recommended first verification step from the repository root is to run the two lightweight host-side unit tests documented in that guide: python -m unittest sarathi-lean/tests/test_vattention_init_dispatch.py and python -m unittest sarathi-lean/tests/test_fragmentation_context_sweep.py. These do not require the full project hardware stack.

If the evaluator has access to the full project container and the required hardware stack, the same guide also documents the higher-confidence container test path and the full end-to-end fragmentation pipeline commands.

References

[1] Microsoft, "vAttention repository," GitHub repository used and extended in this project.
[2] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proceedings of SOSP, 2023.
[3] DeepSeek-AI et al., "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024.
[4] Mistral AI, "Mistral-Nemo-Base-2407," model release and documentation, 2024.
[5] Qwen Team, "Qwen-14B," model release and documentation, 2023.